

Last Time:

□ Intent Inference

□ Intent Expression

Lecture 7

HRI, FALL '26

Andrea Bajcsy

This Time:

□ reward learning → IRL (Inverse Reinforcement learn.)

□ policy learning → BC (Behavior Cloning)

Reward learning vs. Policy learning?

It's often easier to demonstrate good behavior than to manually encode it into a (MDP) model.

ex. in autonomous driving it is hard to write down a reward function that models your personal driving "style".

Assume we are given some trajectory demonstration:

$$\xi_D = (s_0, a_0, s_1, a_1, \dots, s_T, a_T)$$

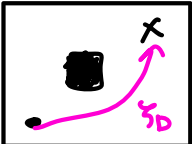
Inverse Reinforcement Learning (IRL) ↗ Both cases of imitation learning ↖ Behavior Cloning (BC)

→ learns a reward function (r) which, when optimized, yields policy that does task

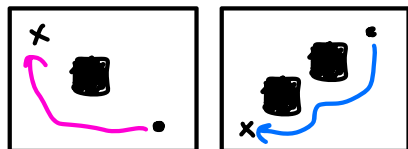
→ seeks to understand human demonstrator's objective, rather than just copying their actions

⊕ "Interpretable" b/c know "what matters"

⊕ Can generalize/adapt to new environments via RL

ex.  learn: $R = \text{mind-the-robot}$

Generalizes:



⊖ Typically needs extra "RL steps" (to discuss later)

→ Directly learns a policy (π) that imitates the human demonstr., mapping states → actions

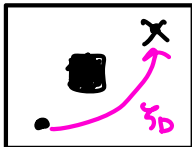
→ Does NOT explicitly infer H's underlying goal; just mimics it

⊕ "Simple", because you have the inputs (states) and the "labels" (actions), no need to infer reward.

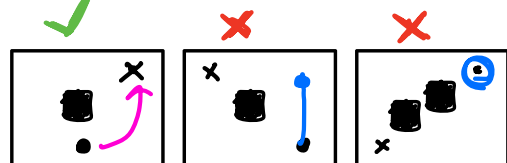
⊕ Computationally cheaper (no extra "RL step")

⊖ compounding errors (more on this later)

⊖ struggles in unseen situations

 learn: $\pi(a|s)$

only imitates!



INVERSE REINFORCEMENT LEARNING - IRL (aka. Inverse optimal control)

Recall: "Forward" Reinforcement Learning

• Given: $s \in S, a \in A$, (sometimes) $P(s'|s, a), r(s, a)$

• Goal: learn $\pi^*(a|s)$ s.t

$$\pi^* := \underset{\pi}{\operatorname{argmax}} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi(s_t)) \right]$$

over $P(s_{t+1}|s_t, \pi)$

Inverse Reinforcement Learning:

• Given: $s \in S, a \in A$, (sometimes) $P(s'|s, a)$, and $\mathcal{D} := \{\zeta_i\}_{i=1}^N$ demonstrations from π^* e.g. Human!

$\zeta_i := (s_0^i, a_0^i, s_1^i, a_1^i, \dots, s_T^i)$

• Goal: learn $r(s, a)$ (+ then use it to learn $\pi^* \dots$)

reward params

Ⓚ But how to search over all reward functions ??

Ⓜ Linear rewards are where alot of foundations started b/c easier to optimize. Assume:

$$r(s) := \theta_1 \phi_1(s) + \theta_2 \phi_2(s) + \dots + \theta_d \phi_d(s)$$

$$= \theta^T \phi(s)$$

reward params / weights $\rightarrow \theta$

"feature vector" e.g. hand design: $\phi(s) = \begin{bmatrix} \text{length} \\ \text{obstacle distance} \\ \text{terrain} \end{bmatrix}$

OR known features from DNN

Ⓢ Turns out value function is linear in "expected features" $\mu^{\pi}(s)$! mean

$$v^{\pi}(s) = \mathbb{E}_P \left[\sum_{t=0}^{\infty} \gamma^t r(s_t) \mid s_0 = s \right] = \theta \cdot \mathbb{E}_P \left[\sum_{t=0}^{\infty} \gamma^t \phi(s_t) \mid s_0 = s \right] = \theta^T \mu^{\pi}(s) !$$

MAX ENTROPY IRL [Ziebart et al, AAAI 2008]

Relaxes the assumption that γ_D is perfect! Instead, it treats demonstrations as observations drawn from some distribution that models the demonstrator as being approximately optimal

↳ SOUND FAMILIAR? ;)

$$(1) \quad P(\gamma | \theta) = \frac{e^{R_\theta(\gamma)}}{\int_{\mathcal{Z}} e^{R_\theta(\bar{\gamma})} d\bar{\gamma}}$$

ASIDE: But... why this distribution? We can get there via Principle of Maximum Entropy (Jaynes, 1957): The probability distribution which best represents your [demonstrator] is the one that leaves you with the largest remaining uncertainty consistent with your evidence (i.e. demonstrations)

... i.e., don't assume anything about your trajectories you don't observe!
OR OCCAM'S RAZOR: "simplest explanation is preferred"
↳ dist. w/ largest entropy (uncertainty) makes least assumptions about the true distribution.

We can actually derive Eqn. (1) by optimizing this principle!

[proof out of scope - see Ziebart's thesis!]

Given demo dataset $\mathcal{D} := \{\gamma_D\}$

$$P^* := \max_P \left\{ \begin{array}{l} \text{def'n of entropy} \\ - \int P(\gamma) \log P(\gamma) d\gamma \end{array} \right\} \quad \left. \begin{array}{l} \text{find distribution } P \text{ that} \\ \text{has maximal entropy} \end{array} \right\}$$
$$\text{s.t. } \mathbb{E}_{\gamma \sim P(\gamma)} [\phi(\gamma)] = \frac{1}{|\mathcal{D}|} \sum_{\gamma \in \mathcal{D}} \phi(\gamma) \quad \left. \begin{array}{l} P \text{ should induce feature counts} \\ \text{that match empirical evidence} \end{array} \right\}$$

P must follow basic prob. laws

$$\left\{ \begin{array}{l} \int P(\gamma) d\gamma = 1 \\ P(\gamma) \geq 0, \forall \gamma \in \mathcal{Z} \end{array} \right.$$

(Back to) Key idea: treat demonstrations as observations drawn from some distribution that models the demonstrator as approximately optimal

$$(1) \quad P(\zeta | \theta) = \frac{e^{R_{\theta}(\zeta)}}{\int e^{R_{\theta}(\bar{\zeta})} d\bar{\zeta}}$$

⇒ let's plug in our linear reward model to get.

$$\textcircled{*} \quad P(\zeta | \theta) = \frac{e^{\theta^T \phi(\zeta)}}{\int e^{\theta^T \phi(\bar{\zeta})} d\bar{\zeta}}$$

← same as before but $R_{\theta}(s) \equiv \theta^T \phi(s)$

② How do we find θ given $\mathcal{D} = \{\zeta_0\}$? ← our data of (one) trajectory

A1 If $\theta \in \{\theta_1, \theta_2, \dots, \theta_N\}$ in (small) discrete space, could use Bayes' rule to obtain exact $P(\theta | \mathcal{D})$ like Lecture 6.

A2 If $\theta \in \mathbb{R}^n$ is continuous then not typically tractable to have exact posterior, so instead compute maximum likelihood estimate (MLE)

MLE: $\max_{\theta} P(\zeta_0 | \theta)$

$$\Leftrightarrow \max_{\theta} \log P(\zeta_0 | \theta) \Rightarrow \max_{\theta} \log \frac{e^{\theta^T \phi(\zeta_0)}}{\int e^{\theta^T \phi(\zeta)} d\zeta}$$

$$\Leftrightarrow \max_{\theta} \underbrace{\theta^T \phi(\zeta_0) - \log \left[\int e^{\theta^T \phi(\zeta)} d\zeta \right]}$$

do gradient ascent again!

$$\nabla_{\theta} : \phi(z_D) - \frac{1}{\int e^{\theta^T \phi(z)} dz} \cdot \int \phi(z) e^{\theta^T \phi(z)} dz$$

$$\Leftrightarrow \phi(z_D) - \int \phi(z) \frac{e^{\theta^T \phi(z)}}{\int e^{\theta^T \phi(\bar{z})} d\bar{z}} dz \quad := P(z|\theta) \text{ by def'n!}$$

$$\Leftrightarrow \phi(z_D) - \int \phi(z) \cdot P(z|\theta) dz \quad := \mathbb{E}[\phi(z)] \text{ expectation!}$$

$z \sim P(z|\theta)$

Max Ent IRL Gradient Ascent:

$$\theta_{i+1} \leftarrow \theta_i + \alpha [\underbrace{\phi(z_D)}_{\substack{\text{observed} \\ \text{feature values} \\ \text{of demonstrator} \\ \text{(human)}}}] - \underbrace{\mathbb{E}[\phi(z)]}_{\substack{\text{expected feature values induced by } \theta_i \text{ (robot)}}}]$$

INTUITION:

→ want features of H's behavior to be indistinguishable from features of R's behavior

Behavior Cloning (BC)

The "simplest" algorithm to run on your dataset $\mathcal{D} := \{z_D\}$

Train a model (e.g. NN) to predict expert actions

given observed states, treating it as if it was supervised

learning:

$$\min_{\theta} \sum_{(s^i, a^i) \in \mathcal{D}} \ell(a, \pi_{\theta}(s))$$

some loss penalizing predicted vs. real expert action.
(e.g. $\|a - \pi(s)\|_2^2$)

looks like supervised learning, trying to match expert @ every state in demonstrations

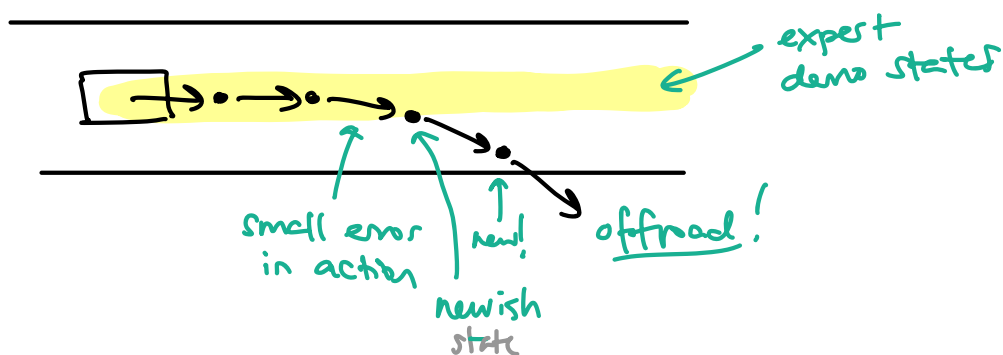
⇒ Issues w/ BC:

In standard SL, we assume that the data pts. are ^{identical} independent & identically distributed (IID) $\uparrow$

... $P_{X_1}(x) = P_{X_k}(x) \forall k, \forall x$
 $\xrightarrow{\text{indep}} P_{X_1, \dots, X_n}(x_1, \dots, x_n) = \prod_{i=1}^n P_{X_i}(x_i)$

However, IL is in a sequential domain where current actions influence future states $\Rightarrow$ assumption is wrong!

$\Rightarrow$ this means BC is prone to error accumulation & can't recover as it drifts away from demonstration distribution
ex. ALVINN (CMU '88)



ASIDE: Theoretical Argument

Suppose supervised learning predictor has a chance ϵ of prediction error. If the data is iid, over T predictions we get total error:

$$P(\text{error}_1, \text{error}_2 \dots \text{error}_T) \stackrel{\text{iid}}{=} P(e_1) + \dots + P(e_T) = T\epsilon.$$

independently identically distributed
 $P(x, y) = P(x) \cdot P(y)$
and
 $P(x) = P(y)$

But, since BC is NOT iid, errors @ one step

impact future errors informal:

$$\leq \epsilon \cdot 1 + \epsilon \cdot 2 + \dots + \epsilon(T-2) + \epsilon(T-1) \stackrel{\text{triangular sum}}{=} \frac{T(T-1)}{2} \cdot \epsilon \approx T^2 \epsilon$$

errors accumulate quadratically!

↑ make mistake @ $t=1$
 ↑ make mistake @ 1 or 2

$$P(\epsilon_1 \cup \epsilon_2) = P(\epsilon_1) + P(\epsilon_2) - P(\epsilon_1 \cap \epsilon_2) \leq \epsilon \cdot 2$$

$$P(\epsilon_1 \cup \epsilon_2 \cup \dots \cup \epsilon_T) \leq P(\epsilon_1) + P(\epsilon_2) + \dots + P(\epsilon_T) = \epsilon \cdot (T-1)$$